ELTE
FACULTY OF
INFORMATICS

PROGRAMMING

Combining PoA's

Zsuzsa Pluhár
pluharzs@inf.elte.hu

Refreshing

Programming patterns

1. Summation
2. Counting
3. Maximum selection
4. Copy
5. Multiple item selection

problems with sum and "all"
↓
counting loop

6. Search
7. Decision
8. Selection

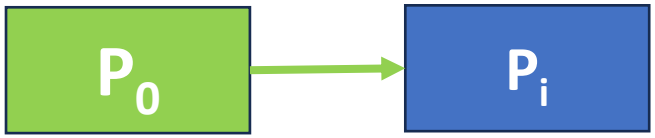
problem with "exists "
↓
conditional loop

More programming patterns

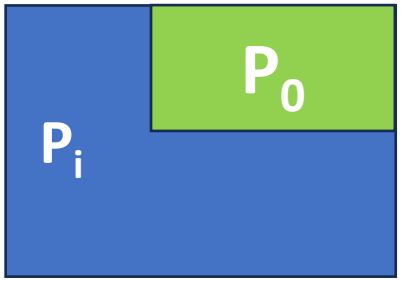
Using more patterns

Complex problems – we need more patterns

- pattern as a function -> combine them



We will use the "result" of the P_0



P_0: output is mostly a Boolean value (a single value)

$\Rightarrow$ decision, search, …

P_i: patterns contain an "attribute function"

More programming patterns

one after the other

Example – books

A library store the borrowings. They would like to know whether proportionally more girls read Terry Pratchett's books than boys read Neil Gaiman's.

Example – books

A library store the borrowings. They would like to know whether proportionally more girls read Terry Pratchett's books than boys read Neil Gaiman's.

Book	Sex
T.Pratchett	G
N.Gaiman	G
N.Gaiman	B
T.Pratchett	G
T.Pratchett	B
T.Pratchett	G
N.Gaiman	G
T.Pratchett	G
N.Gaiman	B
N.Gaiman	B
T.Pratchett	G
T.Pratchett	G
N.Gaiman	B

1

→ T.Pratchett 7
→ N.Gaiman 6
→ T.Pratchett and girl 6
→ N.Gaiman and boy 4

4 "counting"

2

→ T.Pratchett: 6/7
→ N.Gaiman: 4/6

3

→ true

Example – books

A library store the borrowings. They would like to know whether proportionally more girls read Terry Pratchett's books than boys read Neil Gaiman's.

Specification:

```
In: n∈N, readers∈Tr[1..n], Tr=(book:S x sex:C)
Out: gaimanb∈L
Aux: cnttp∈N, cntng∈N, cntg∈N, cntb∈N
Pre: ∀i∈[1..n]:(readers[i].sex='G' or readers[i].sex='B')
Post:
  cnttp = COUNT(i=1..n, readers[i].book="T.Pratchett") and
  cntng = COUNT(i=1..n, readers[i].book="N.Gaiman")    and
  cntg  = COUNT(i=1..n, readers[i].book="T.Pratchett"  and
                        readers[i].sex="G") and
  cntb  = COUNT(i=1..n, readers[i].book ="N.Gaiman"  and
                        readers[i].sex="B") and
  gaimanb=(cng / cnttp) > (cntb / cntng)
```

1

3 2

Example – books

Specification: pattern: `cnt=COUNT(i=b..e, A(i))`

Post:

```
cnttp = COUNT(i=1..n, readers[i].book="T.Pratchett") and
cntng = COUNT(i=1..n, readers[i].book="N.Gaiman")    and
cntg  = COUNT(i=1..n, readers[i].book="T.Pratchett"  and
                      readers[i].sex="G") and
cntb  = COUNT(i=1..n, readers[i].book ="N.Gaiman"  and
                      readers[i].sex="B") and
```

1

```
cnt    ~  cnttp
b..e   ~  1..n
A(i)   ~  readers[i].book="T.Pratchett"
```

```
cnt    ~  cntg
b..e   ~  1..n
A(i)   ~  readers[i].book="T.Pratchet
          t"  and readers[i].sex="G")
```

```
cnt    ~  cntng
b..e   ~  1..n
A(i)   ~  readers[i].book="N.Gaiman"
```

```
cnt    ~  cntb
b..e   ~  1..n
A(i)   ~  readers[i].book ="N.Gaiman"
          and readers[i].sex="B"
```

Example – books

```
cnt:=0
i=b..e
    A(i)
    T / F
    cnt:=cnt+1 | -
```

cnt	~	cnttp
b..e	~	1..n
A(i)	~	readers[i].book="T.Pratchett"
cnt	~	cntng
b..e	~	1..n
A(i)	~	readers[i].book="N.Gaiman"
cnt	~	cntg
b..e	~	1..n
A(i)	~	readers[i].book="T.Pratchett" and readers[i].sex="G")
cnt	~	cntb
b..e	~	1..n
A(i)	~	readers[i].book ="N.Gaiman" and readers[i].sex="B"

```
cntp:=0
i=1..n
    readers[i].book="T.Pratchett"
    T / F
    cntp:=cntp+1 | -
cntng:=0
i=1..n
    readers[i].book="N.Gaiman"
    T / F
    cntng:=cntng+1 | -
cntg:=0
i=1..n
    readers[i].book="T.Pratchett" and
    readers[i].sex="G"
    T / F
    cntg:=cntg+1 | -
cntb:=0
i=1..n
    readers[i].book="N.Gaiman" and
    readers[i].sex="B"
    T / F
    cntb:=cntb+1 | -
gaimanb:=(cntg/cnttp)>(cntb/cntng)
```

Example – books

Conclusions:

- in specification one after the other
- in algorithm too – sequence of "instructions"
- helper/auxiliary variable

input

Book	Sex
T.Pratchett	G
N.Gaiman	G
N.Gaiman	B
T.Pratchett	G
T.Pratchett	B
T.Pratchett	G
N.Gaiman	G
T.Pratchett	G
N.Gaiman	B
N.Gaiman	B
T.Pratchett	G
T.Pratchett	G
N.Gaiman	B

1

→ T.Pratchett = 7
→ N.Gaiman = 6
→ T.Pratchett and girl = 6
→ N.Gaiman and boy = 4

helper variable

2

→ T.Pratchett: 6/7
→ N.Gaiman: 4/6

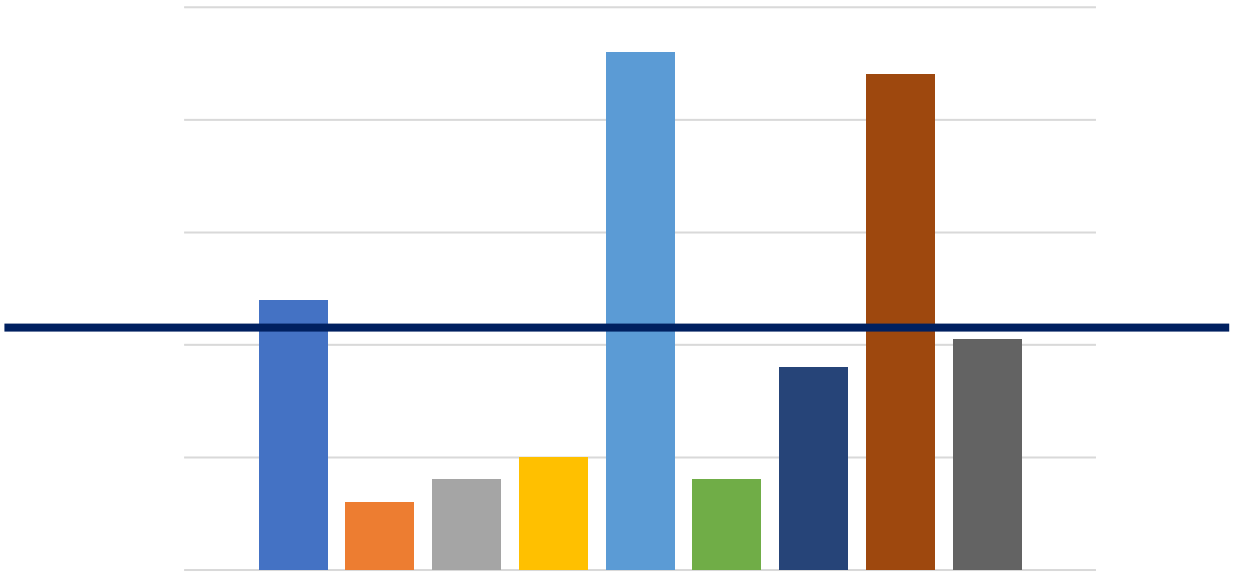
3

→ true

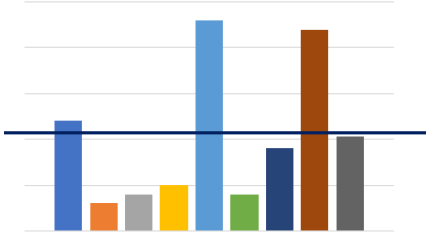
output

Example – grades

We store the grades of "n" students. Let's list the students over the average grade.



Example – grades



We store the grades of "n" students. Let's list the students over the average grade.

grades
5
3
4
2
2
4
2
1

→ average: 3.25

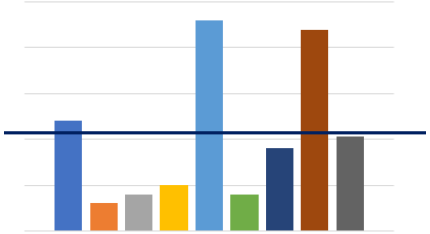
summation

grades
5
3
4
2
2
4
2
1

students
1
3
6

multiple item selection

Example – grades



We store the grades of "n" students. Let's list the students over the average grade.

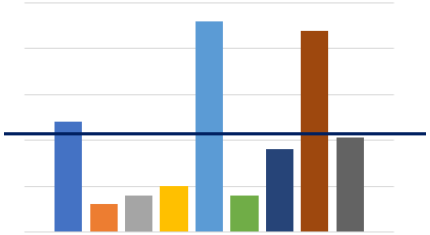
Specification:

```
In: n∈N, grades∈N[1..n]
Out: cnt∈N, students∈N[1..cnt]
Aux: av∈R
Pre: ∀i∈[1..n]:(1<=grades[i]<=5)
Post: av=SUM(i=1..n,grades[i])/n and
      (cnt, students)=MIS(i=1..n,grades[i]>av,i)
```

```
s=SUM(i=b..e, f(i))
```

```
(cnt,y)=MIS(i=b..e,A(i),f(i))
```

Example – grades



```
Post: av=SUM(i=1..n,grades[i])/n and
```

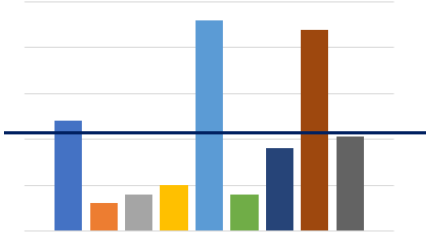
```
s=SUM(i=b..e, f(i))
```

s	~	av*n
b..e	~	1..n
A(i)	~	grades[i]

```
s:=0
i=b..e
    s:=s+f(i)
```

```
s:=0
i=1..n
    s:=s+grades[i]
av:=s/n
```

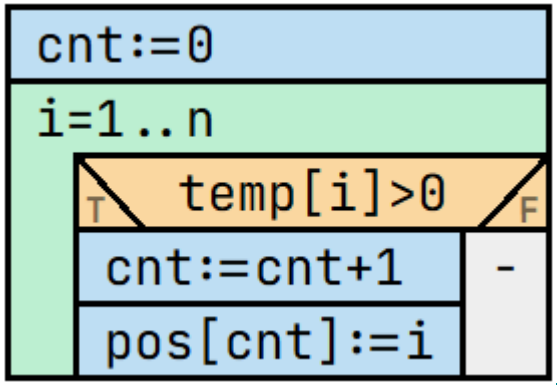
Example – grades

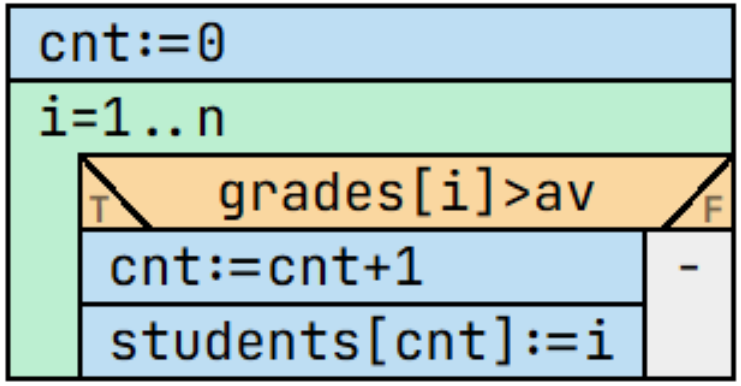


```
Post: ... and
      (cnt, students)=MIS(i=1..n,grades[i]>av,i)
```

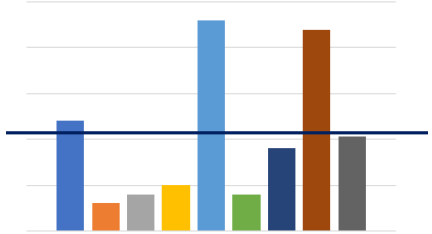
```
(cnt,y)=MIS(i=b..e,A(i),f(i))
```

cnt	~	cnt
b..e	~	1..n
A(i)	~	grades[i]>av
f(i)	~	i



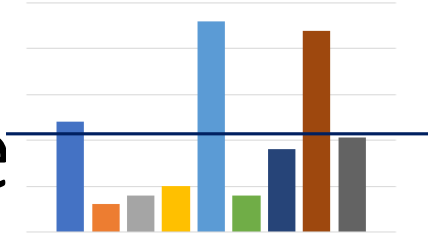


Example – grades



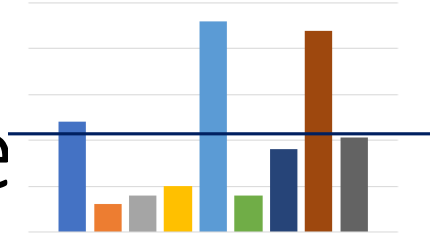
```
s:=0
i=1..n
    s:=s+grades[i]
av:=s/n
cnt:=0
i=1..n
    grades[i]>av
    T: cnt:=cnt+1
       students[cnt]:=i
    F: -
```

Example – grades, the code



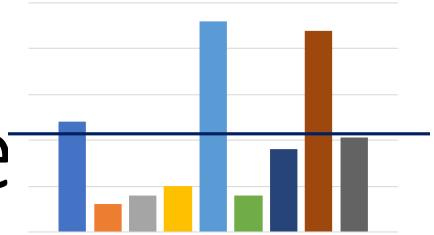
```
static void Main(string[] args) {
  // declaring (variables, specification: in/out)
  int n;
  int[] grades;
  double av;
  int cnt;
  List<int> students = new List<int>();

  // Reading the input
  readIn(out n,out grades);
  av=average(n,grades);
  (cnt,students)=mis_students(n,grades,av);
  writeOut(cnt,students);
}
```

Example – grades, the code



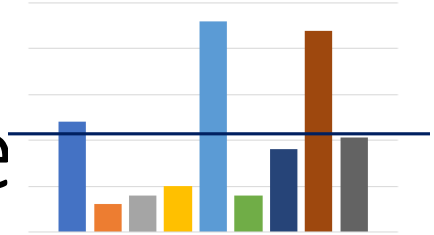
```
static void Main(string[] args) {

  int n;
  int[] grades;
  double av;
  int cnt;
  List<int> students = new List<int>();

  readIn(out grades);
  av=average(grades);
  students=mis_students(grades,av);
  writeOut(students);
}
```

Example – grades, the code



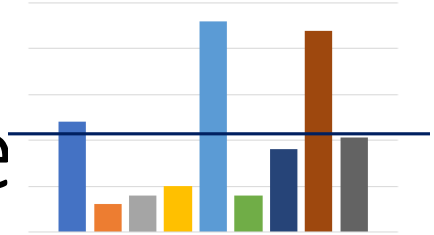
```
static void readIn(out int[] x) {
    int n;
    Console.Error.Write("n = ");
    int.TryParse(Console.ReadLine(), out n);
    x = new int[n];
    for (int i = 0; i < n; i++) {
        Console.Error.Write("{0}. grade = ", i + 1);
        int.TryParse(Console.ReadLine(), out x[i]);
    }
}
static void writeOut(List<int> x) {
    for (int i = 0; i < x.Count; i++) {
        Console.Error.Write("{0}. student; ", x[i] + 1);
    }
}
```

Example – grades, the code



```
static double average(int[] x) {
    int sum = 0;
    for (int i=0;i<x.Length;i++) {
        sum += x[i];
    }
    double sav = (double)sum / x.Length;
    return sav;
}
static List<int> mis_students(int[] x,double y) {
   List<int> st = new List<int>();
   for (int i= 0; i < x.Length; i++) {
        if (x[i]>y) {
            st.Add(i);
        }
    }
    return st;
}
```

Example – grades, the code

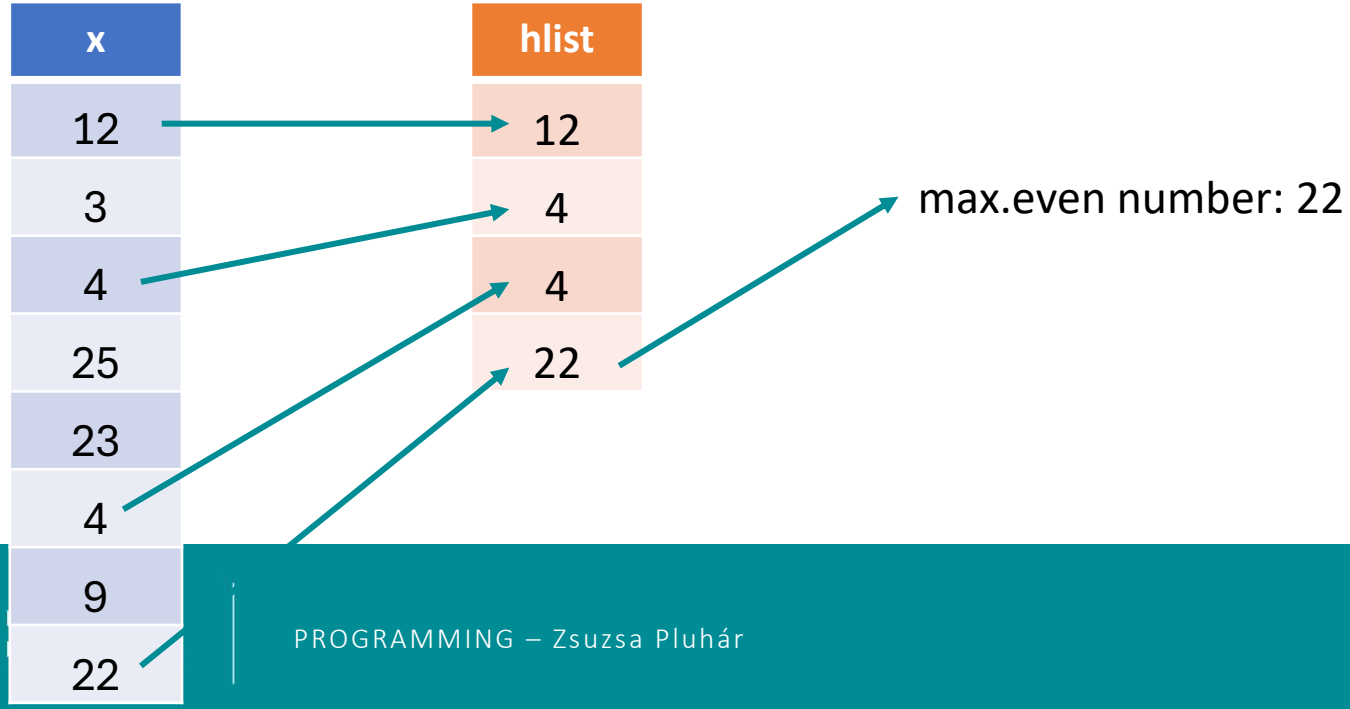


```
static void Main(string[] args) {
    int[] grades;
    List<int> students = new List<int>();

    readIn(out grades);
    students = mis_students(grades,average(grades));
    writeOut(students);
}
```

Example – conditional maximum selection

Find the highest even number in an integer sequence

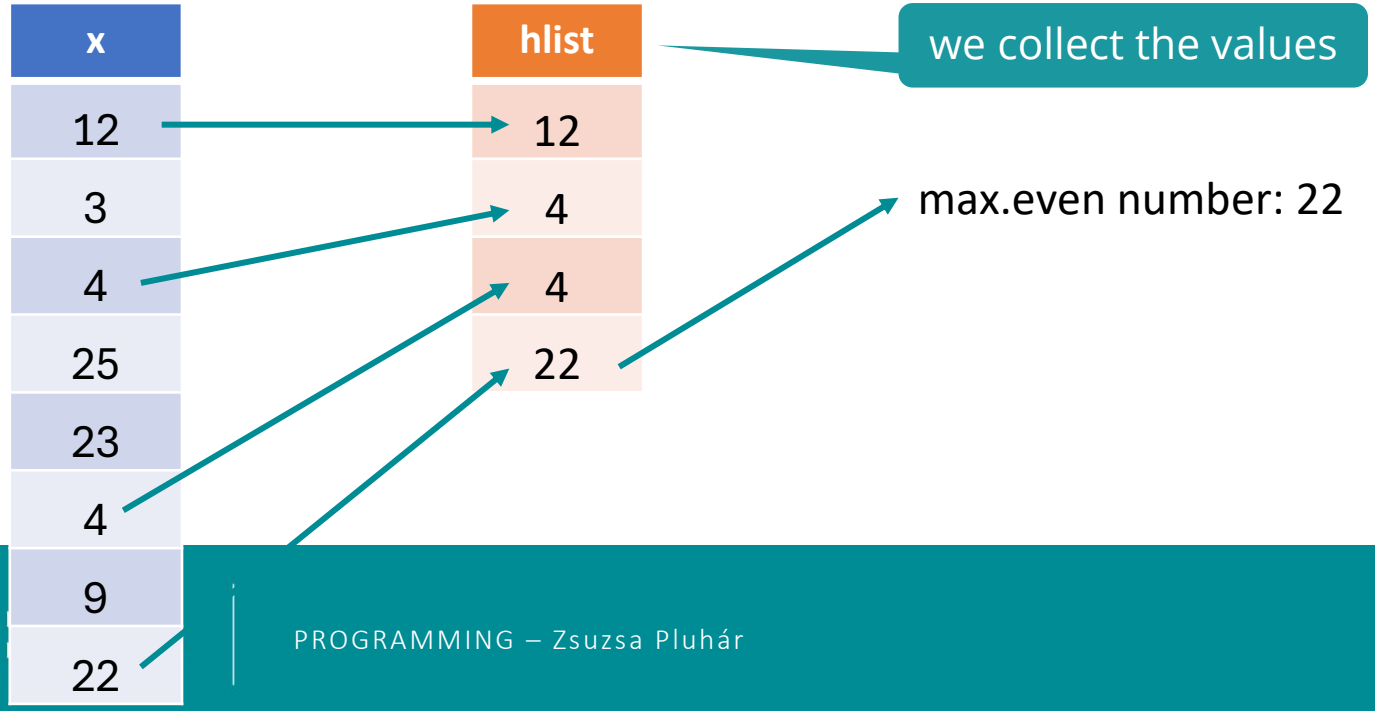
Solution$_1$:
- list/collect all even numbers –> MIS
- from this list find the maximum –> Maximum selection



Example – conditional maximum selection

Find the highest even number in an integer sequence

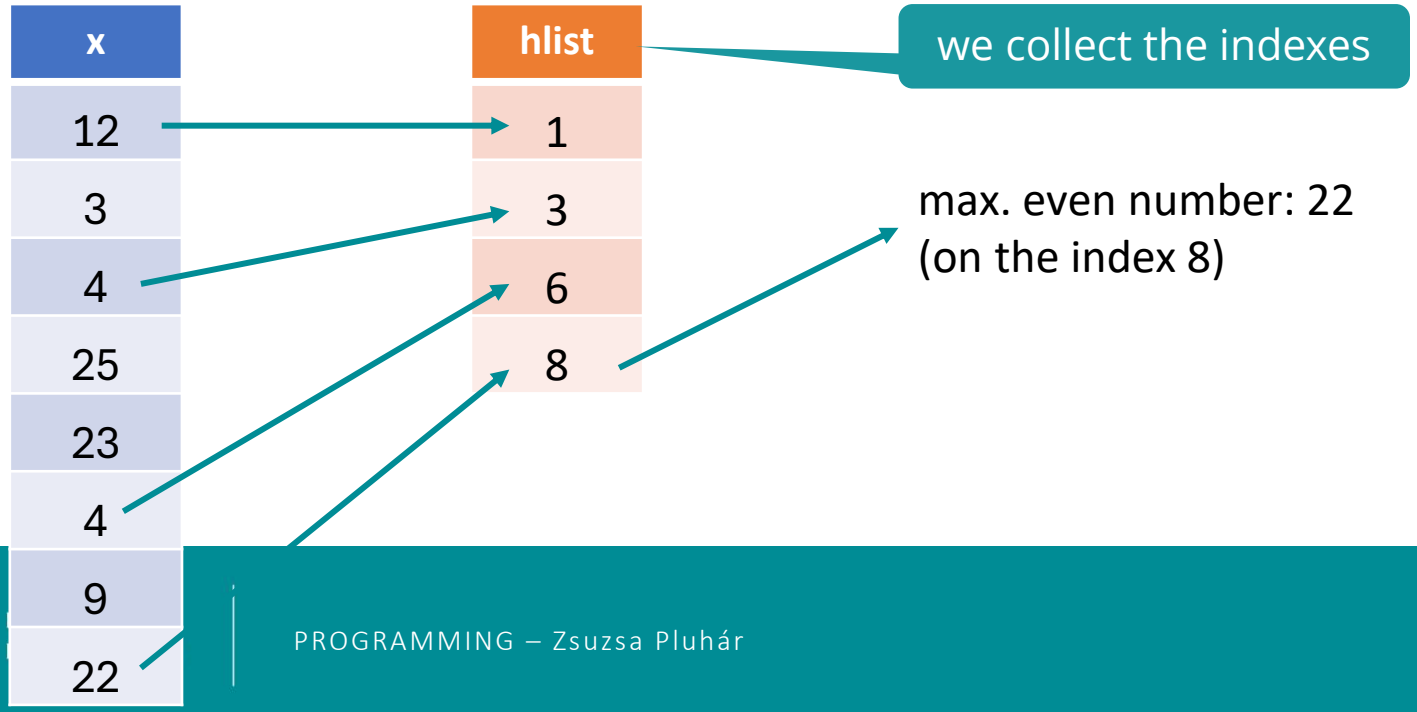
Solution$_1$:
- list/collect all even numbers –> MIS
- from this list find the maximum –> Maximum selection



Example – conditional maximum selection

Find the highest even number in an integer sequence

Solution$_1$:
- list/collect all even numbers –> MIS
- from this list find the maximum –> Maximum selection



Example – conditional maximum selection

Find the highest even number in an integer sequence

Specification:

```
In: n∈N, x∈Z[1..n]
Aux: cnt∈N, hlist∈Z[1..cnt]
Out: maxeven∈Z
Pre: -
Post: (cnt,hlist)=MIS(i=1..n,even(x[i]),x[i]) and
      (,maxeven)=MAX(i=1..cnt,hlist[i])
```

```
(cnt,y)=MIS(i=b..e,A(i),f(i))
```

```
(maxind,maxval)=MAX(i=b..e, f(i))
```

Example – conditional maximum selection

Find the highest even number in an integer sequence

Specification:

```
In: n∈N, x∈Z[1..n]
Aux: cnt∈N, hlist∈Z[1..cnt]
Out: maxeven∈Z
Fn: even:Z->L, even(a)=(a mod 2 = 0)
Pre: -
Post: (cnt,hlist)=MIS(i=1..n,even(x[i]),x[i]) and
      (,maxeven)=MAX(i=1..cnt,hlist[i])
```

A(i) → A(f(i))

Example – conditional maximum selection

Find the highest even number in an integer sequence

Specification$_{\textbf{indexes}}$:

```
In: n∈N, x∈Z[1..n]
Aux: cnt∈N, hlist∈N[1..cnt]
Out: maxeven∈Z
Fn: even:Z->L, even(a)=(a mod 2 = 0)
Pre: -
Post: (cnt,hlist)=MIS(i=1..n,even(x[i]),i) and
      (,maxeven)=MAX(i=1..cnt,x[hlist[i]])
```

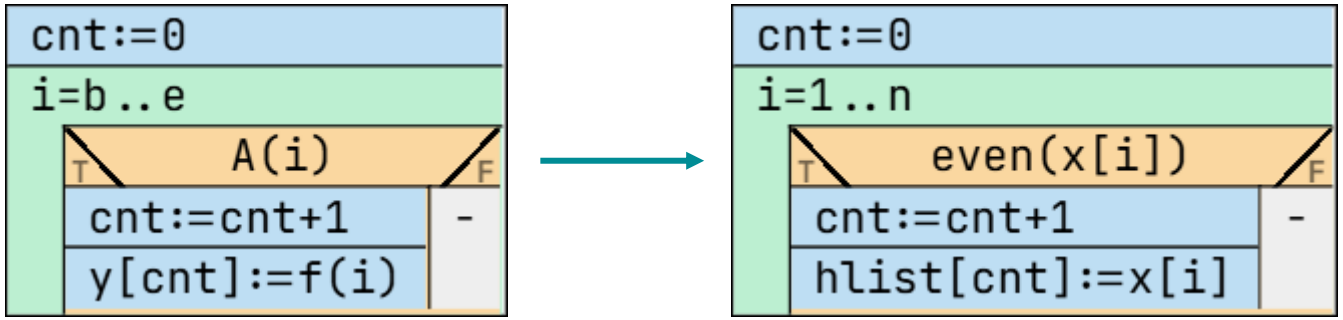
Example – conditional maximum selection

```
Post: (cnt,hlist)=MIS(i=1..n,even(x[i]),x[i])
```

cnt, y	~	cnt, hlist
b..e	~	1..n
A(i)	~	even(x[i])
f(i)	~	x[i]

```
(cnt,y)=MIS(i=b..e,A(i),f(i))
```

Algorithm:



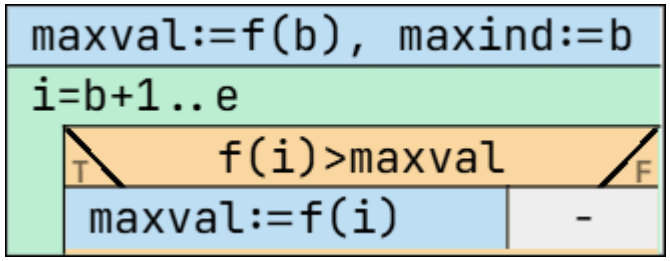
Example – conditional maximum selection

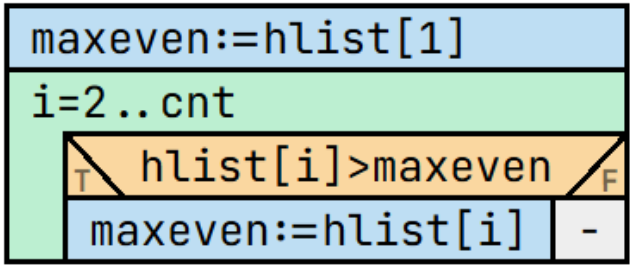
```
Post: (,maxeven)=MAX(i=1..cnt,hlist[i])
```

```
(maxind,maxval)=MAX(i=b..e, f(i))
```

maxind	~	-
maxval	~	maxeven
b..e	~	1..cnt
f(i)	~	hlist[i]

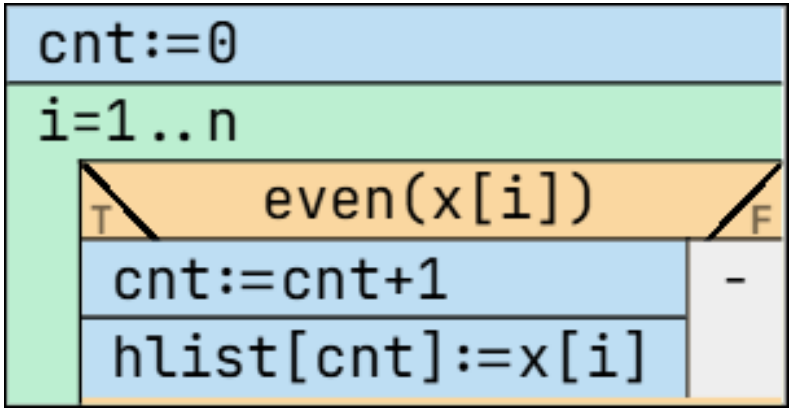
Algorithm:

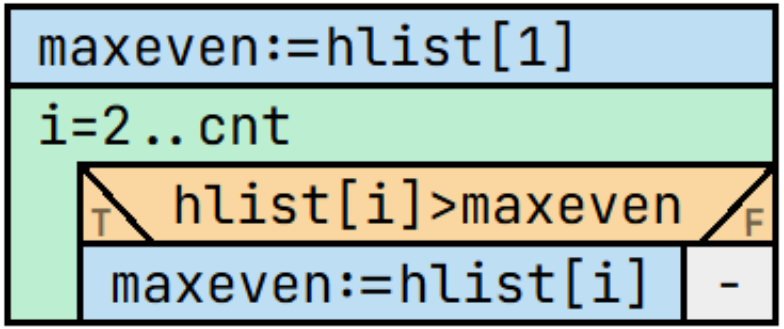




Example – conditional maximum selection

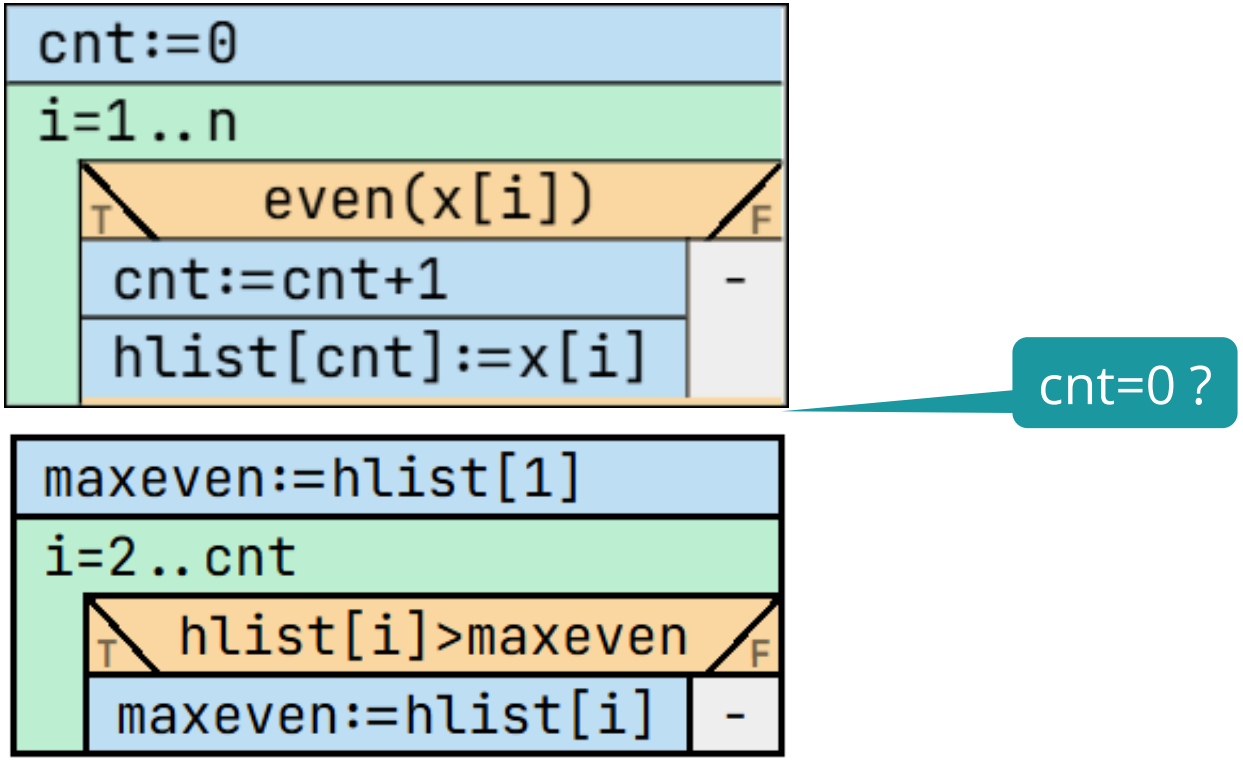
Algorithm:





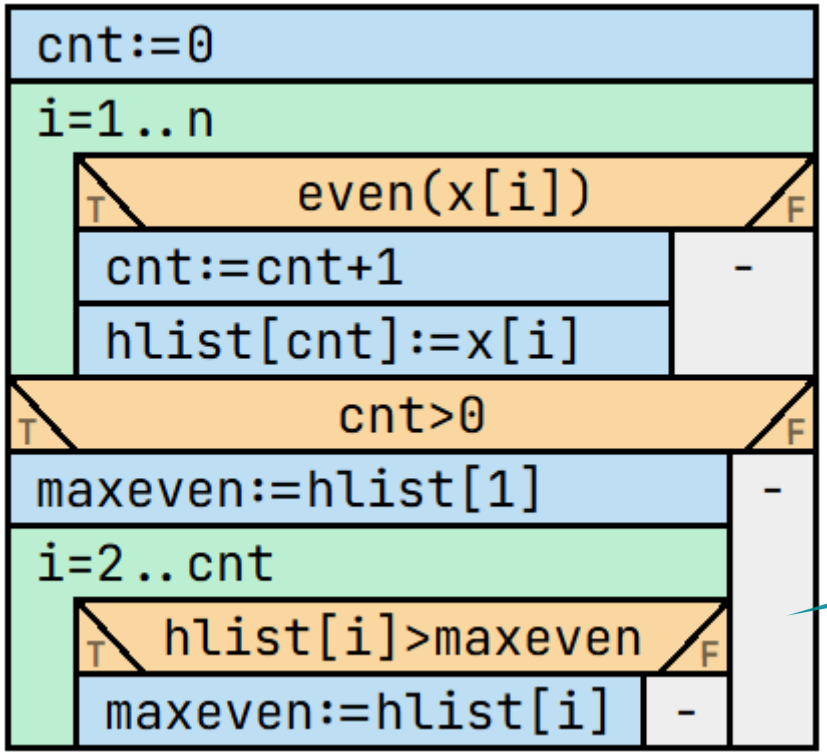
Example – conditional maximum selection

Algorithm:



Example – conditional maximum selection

Algorithm:



Out: exists∈L

Example – conditional maximum selection

Find the highest even number in an integer sequence

- Create the algorithm for "collecting the indexes in the hlist"
- Create the algorithm of the even function
- Code the solution (with functions)

Example – conditional maximum selection$_2$

Find the highest even number in an integer sequence

Solution$_2$:

- find the first even number –> Search
- from this item find the maximum –> Maximum selection

x
11
3
4
25
23
4
9
22

first even number:
exists: true
index: 3

max.even number: 22

Example – conditional maximum selection$_2$

Find the highest even number in an integer sequence

Specification:

```
In: n∈N, x∈Z[1..n]
Aux: ind∈N
Out: exists∈L, maxeven∈Z
Fn: even:Z->L, even(a)=(a mod 2 = 0)

Pre: -
Post: (exists,ind)=SEARCH(i=1..n,even(x[i])) and
      exists->(,maxeven)=MAX(i=ind..n,x[i])
```

Example – conditional maximum selection$_2$

Find the highest even number in an integer sequence

Specification:

```
In: n∈N, x∈Z[1..n]
Aux: ind∈N
Out: exists∈L, maxeven∈Z
Fn: even:Z->L, even(a)=(a mod 2 = 0)

Pre: -
Post: (exists,ind)=SEARCH(i=1..n,even(x[i])) and
      exists->(,maxeven)=MAX(i=ind..n,x[i])
```

```
(exists,ind)=SEARCH(i=b..e,A(i))
```

```
(maxind,maxval)=MAX(i=b..e, f(i))
```

Example – conditional maximum selection$_2$

Find the highest even number in an integer sequence

Specification:

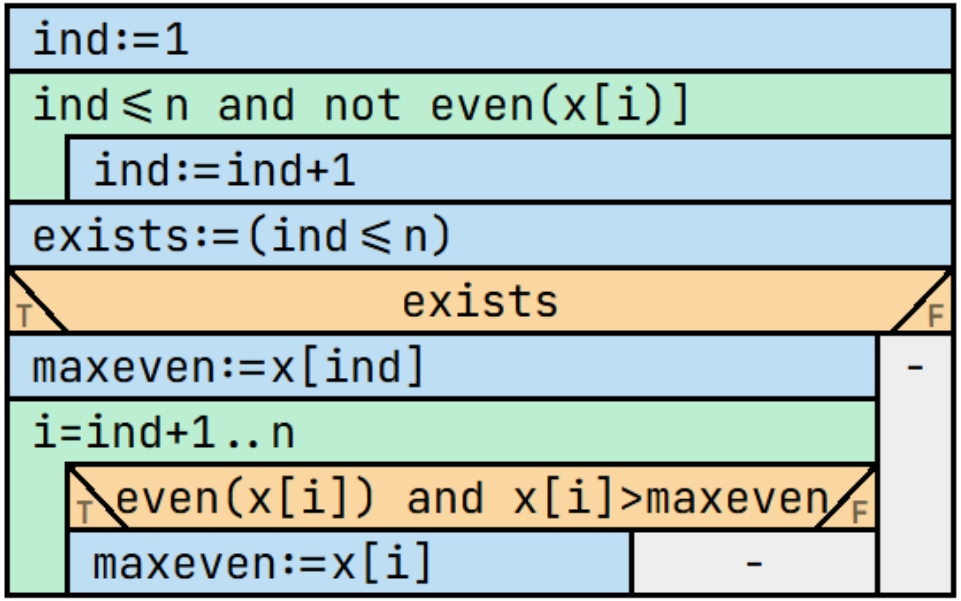
```
In: n∈N, x∈Z[1..n]
Aux: ind∈N
Out: exists∈L, maxeven∈Z
Fn: even:Z->L, even(a)=(a mod 2 = 0)
Fn: g:Z->Z, g(a)={a, if even(a); ? else}
Pre: -
Post: (exists,ind)=SEARCH(i=1..n,even(x[i])) and
      exists->(,maxeven)=MAX(i=ind..n,g(x[i]))
```

Example – conditional maximum selection$_2$

```
(exists,ind)=SEARCH(i=1..n,even(x[i]))   and
exists->(,maxeven)=MAX(i=ind..n,g(x[i]))
```

exists, ind	~	exists, ind
b..e	~	1..n
f(i)	~	even(x[i])

maxind, maxval	~	-, maxeven
b..e	~	ind..n
f(i)	~	g(x[i])



Example – conditional maximum selection$_2$

```
(exists,ind):= search_first_even(x)
T ─────────── exists ─────────── F
maxeven:=max_sel(ind,x)   |   -
```

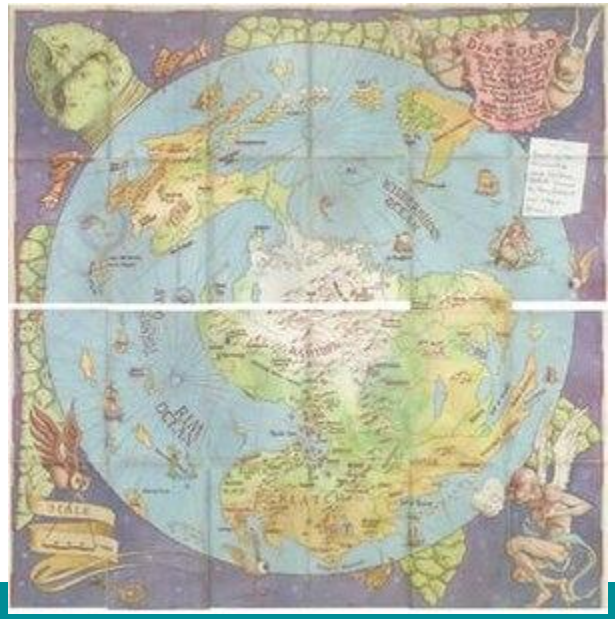
```
search_first_even(xx:Array[1..:Integer]):(Boolean,Integer)
ind:=1
ind ≤ length(xx) and not even(xx[i])
  ind:=ind+1
exists:=(ind ≤ length(xx))
search_first_even:=(exists,ind)
```

```
max_sel(ind:Integer,xx:Array[1..:Integer]):Integer
maxeven:=xx[ind]
i=ind+1..length(xx)
  T ─────── x[i]>maxeven ─────── F
  maxeven:=xx[i]   |   -
max_sel:=maxeven
```

More programming patterns
nested in each other

Example - intersection

The Librarian stores the locations that were visited by the Chair of the Indefinite Studies and the Lecturer in Recent Runes. He wants to make a list of places where both wizards have been?



Example - intersection

Task: We have two sequences and would like to list all items that can be found in both.

C.I.S.
Ankh-Morpork
Klatch
Überwald
Pseudopolis

L.R.R.
Holy Wood
Ankh-Morpork
Quirm
Lancre
Überwald

both
Ankh-Morpork
Überwald

Example - intersection

Task: We have two sequences and would like to list all items that can be found in both.

C.I.S.
Ankh-Morpork
Klatch
Überwald
Pseudopolis

L.B.R.
Holy Wood
Ankh-Morpork
Quirm
Lancre
Überwald

both
Ankh-Morpork
Überwald

List all item with a given attribute –> **multiple item selection** (MIS)

Example - intersection

Task: We have two sequences and would like to list all items that can be found in both.

Specification:

```
In: n∈N, m∈N, cis∈S[1..n], lrr∈S[1..m]
Out: cnt∈N, both∈S[1..cnt]
Pre: -
Post:
(cnt,both)=MIS(i=1..n,isin(cis[i],lrr),cis[i])
```

The isin function contains the „subtask"

we collect the values

Example - intersection

Task: We have two sequences and would like to list all items that can be found in both.

try to decide wheter a sequence (array, list) contains an item –> decision

Specification (isin):

```
In: m∈N, lrr∈S[1..m], what∈S
Out: exists∈L
Pre: -
Post:exists=EXISTS(j=1..m,what=lrr[j])
```

Example - intersection

Task: We have two sequences and would like to list all items that can be found in both.

Specification:

m is global

```
In: n∈N, m∈N, cis∈S[1..n], lrr∈S[1..m]
Out: cnt∈N, both∈S[1..cnt]
Fn: isin: S x S[] -> L,
    isin(what,lrr)=EXISTS(j=1..m,what=lrr[j])
Pre: -
Post:
(cnt,both)=MIS(i=1..n,isin(cis[i],lrr),cis[i])
```

Example - intersection

Task: We have two sequences and would like to list all items that can be found in both.

Specification:

```
In: n∈N, m∈N, cis∈S[1..n], lrr∈S[1..m]
Out: cnt∈N, both∈S[1..cnt]
Fn: isin: S x S[] -> L,
    isin(what,lrr)=EXISTS(j=1..length(lrr),what=lrr[j])
Pre: -
Post: (cnt,both)=MIS(i=1..n,isin(cis[i],lrr),cis[i])
```

we can „get" the length of a sequence (array)

Example - intersection

MIS pattern:

```
In: b∈Z, e∈Z
Out: cnt∈N, y∈H[1..cnt]
Pre: -
Post: (cnt,y)=MIS(i=b..e, A(i),f(i))
```

decision pattern:

```
In: b∈Z, e∈Z
Out: exists∈L
Pre: -
Post: exists=EXISTS(i=b..e, A(i))
```

Specific task:

```
In: n∈N, m∈N, cis∈S[1..n], lrr∈S[1..m]
Out: cnt∈N, both∈S[1..cnt]
Fn: isin: S x S[] -> L,
  isin(what,lrr)=
  EXISTS(j=1..length(lrr),what=lrr[j])
Pre: -
Post: (cnt,both)=
  MIS(i=1..n,isin(cis[i],lrr),cis[i])
```

MIS		
cnt, y	~	cnt, both
b..e	~	1..n
A(i)	~	isin(cis[i],lrr)

decision		
i	~	j
b..e	~	1..m (length(lrr))
A(i)	~	what=lrr[j]

Example - intersection

2 separated tasks.

Patterns

```
cnt:=0
i=b..e
    A(i)  T / F
    cnt:=cnt+1 | -
    y[cnt]:=f(i)
```

```
i:=b
i≤e and not A(i)
    i:=i+1
exists:=(i≤e)
```

Algorithm

```
cnt:=0
i=1..n
    isin(cis[i],lrr)  T / F
    cnt:=cnt+1 | -
    both[cnt]:=cis[i]
```

```
j:=1
j≤m and not (what=lrr[j])
    j:=j+1
exists:=(j≤m)
```

m=length(lrr)

MIS

cnt, y	~	cnt, both
b..e	~	1..n
A(i)	~	isin(cis[i],lrr)

decision

i	~	j
b..e	~	1..m (length(lrr))
A(i)	~	what=lrr[j]

Example - intersection

Algorithm

connecting:

1. **calling a function**: the nested algorithm will be a function, and the "outer" algorithm call it
2. **integration**: copy the nested algorithm

```
cnt:=0
i=1..n
  isin(cis[i],lrr)
  T: cnt:=cnt+1
     both[cnt]:=cis[i]
  F: -
```

```
j:=1
j ≤ m and not (what=lrr[j])
  j:=j+1
exists:=(j ≤ m)
```

Example - intersection

Algorithm

connecting:

1. **calling a function**: the nested algorithm will be a function, and the "outer" algorithm call it
2. **integration**: copy the nested algorithm

```
cnt:=0
i=1..n
  isin(cis[i],lrr)
  T: cnt:=cnt+1
     both[cnt]:=cis[i]
  F: -
```

FOLLOW THIS!

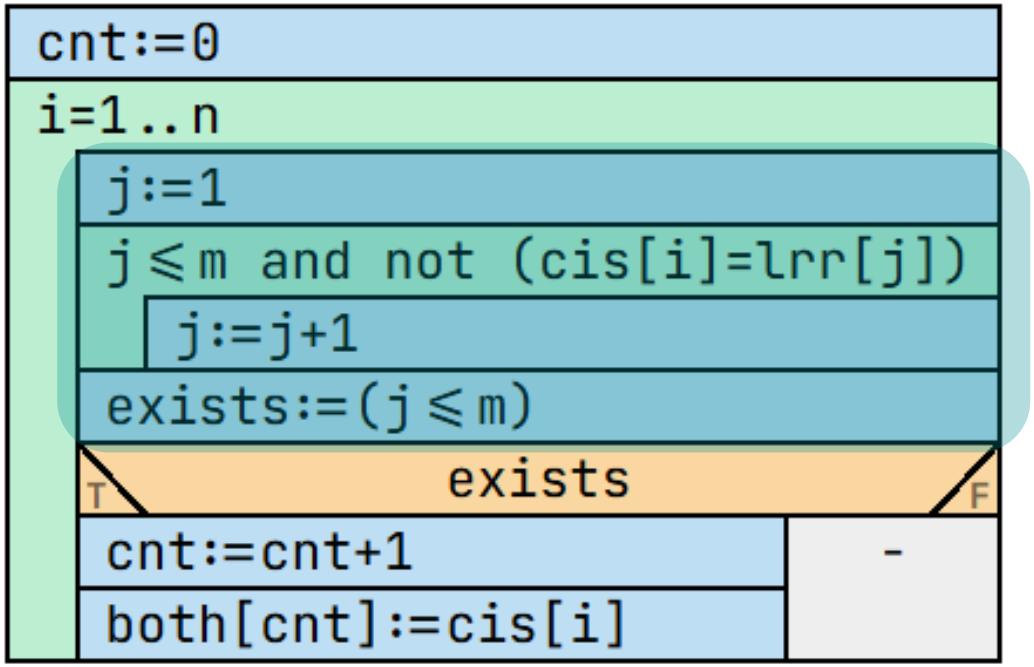
```
isin(what:String, where:String[]):Boolean
j:=1
j ≤ m and not (what=lrr[j])
  j:=j+1
exists:=(j ≤ m)
```

Example - intersection

connecting:

1. **calling a function**: the nested algorithm will be a function, and the "outer" algorithm call it
2. **integration**: copy the nested algorithm

Algorithm



Example – intersection, the code

```
static void Main(string[] args) {
    string[] cis; string[] rrl;
    List<string> bothl = new List<string>();

    readIn(out cis, "Chair of the Indefinite Studies");
    readIn(out rrl, "Lecturer in Recent Runes");
    bothl = mis_both(cis,rrl);
    writeOut(bothl);
}
```

Example – intersection, the code

```
static void readIn(out string[] x, string text) {
     int n;
     Console.Error.Write("n of {0}= ",text);
     int.TryParse(Console.ReadLine(), out n);
     x = new string[n];
     for (int i = 0; i < n; i++) {
         Console.Error.Write("{0}. place = ", i + 1);
        x[i] = Console.ReadLine();
    }
}
static void writeOut(List<string> x) {
    for (int i = 0; i < x.Count; i++) {
        Console.Error.Write("{0}. place; ", x[i]);
    }
}
```

call 2 times: text give extra information

Example – intersection, the code

```
static List<string> mis_both(string[] x1, string[] x2) {
    List<string> st = new List<string>();
    for (int i= 0; i < x1.Length; i++) {
        if (isIn(x1[i],x2)) {
            st.Add(x1[i]);
        }
    }
    return st;
}
static bool isIn(string what, string[] where) {
    int j = 0;
    while (j < where.Length &&! where[j].Equals(what)){
        j++;
    }
    return (j < where.Length);
}
```

what<>where[j])

Example
most frequent value

Spotify stores which Beatles songs started playing when. Let's find out which song is the most frequently played.

1	Here Comes The Sun
2	Hey Jude
3	Twist and Shout
4	I Wanna Hold Your Hand
5	Hey Jude
6	Here Comes The Sun
7	Let It Be
8	Hey Jude
9	Let It Be
10	Hey Dude

mf="Hey Jude"

Example
most frequent value

Let's find out which value occurred most often!

Idea:

1. Record separately how many times each value was played! -> Problematic: new or old, you need to determine!
2. A maximum must be determined: how many times the value occurs (counting), the maximum of the number (maximum selection), and what is the maximum value!

1	Here Comes The Sun
2	Hey Jude
3	Twist and Shout
4	I Wanna Hold Your Hand
5	Hey Jude
6	Here Comes The Sun
7	Let It Be
8	Hey Jude
9	Let It Be
10	Hey Dude

a **counting** in a **maximum selection**

Example
most frequent value

Let's find out which value occurred most often! A maximum must be determined: how many times the value occurs (counting), the maximum of the number (maximum selection), and what is the maximum value!

1	Here Comes The Sun
2	Hey Jude
3	Twist and Shout
4	I Wanna Hold Your Hand
5	Hey Jude
6	Here Comes The Sun
7	Let It Be
8	Hey Jude
9	Let It Be
10	Hey Dude

Specification:

```
In: n∈N, btls∈S[1..n]
Aux: maxind∈N, maxcnt∈N
Out: mf∈S
Fn: hm:N->N, hm(i)=COUNT(j=1..n,btls[i]=btls[j])
Pre: -
Post: (maxind,maxcnt)=MAX(i=1..n,hm(i)) and
    mf=btls[maxind]
```

maxvalue (maxdb) will show the number – we need the value of the input sequence

Example
most frequent value

Maximum selection:

```
In: b∈Z, e∈Z
Out: maxind∈Z, maxval∈H
Pre: e<=u
Post: (maxind,maxval)=
            MAX(i=b..e,f(i))
```

Counting:

```
In: b∈Z, u∈Z
Out: cnt∈N
Pre: -
Post: cnt=COUNT(i=b..e, A(i))
```

Specification:

```
In: n∈N, btls∈S[1..n]
Aux: maxind∈N, maxcnt∈N
Out: mf∈S
Fn: hm:N->N,
    hm(i)=COUNT(j=1..n,btls[i]=btls[j])
Pre: -
Post: (maxind,maxcnt)=MAX(i=1..n,hm(i)) and
    mf=btls[maxind]
```

maximum selection		
maxval	~	maxcnt
b..e	~	1..n
f(i)	~	hm(i)

counting		
i	~	j
b..e	~	1..n
A(i)	~	btls[i]=btls[j]

Example
most frequent value

Maximum selection:

```
maxind:=b, maxval:=f(b)
i=b+1..e
  f(i)>f(maxind)
  T: maxind:=i
     maxval:=f(i)
  F: -
```

maximum selection		
maxval	~	maxcnt
b..e	~	1..n
f(i)	~	hm(i)

```
maxind:=1, maxcnt:=hm(b)
i=2..n
  hm(i)>hm(maxind)
  T: maxind:=i
     maxcnt:=hm(i)
  F: -
mf:=btls[maxind]
```

Counting:

```
cnt:=0
i=b..e
  A(i)
  T: cnt:=cnt+1
  F: -
```

counting		
i	~	j
b..e	~	1..n
A(i)	~	btls[i]=btls[j]

```
hm(i:Integer):Integer
cnt:=0
j=1..n
  btls[i]=btls[j]
  T: cnt:=cnt+1
  F: -
hm:=cnt
```

Example
most frequent value

Variation: only count the number of elements with the same value after a given element.

1	Here Comes The Sun
2	Hey Jude
3	Twist and Shout
4	I Wanna Hold Your Hand
5	Hey Jude
6	Here Comes The Sun
7	Let It Be
8	Hey Jude
9	Let It Be
10	Hey Dude

Specification:

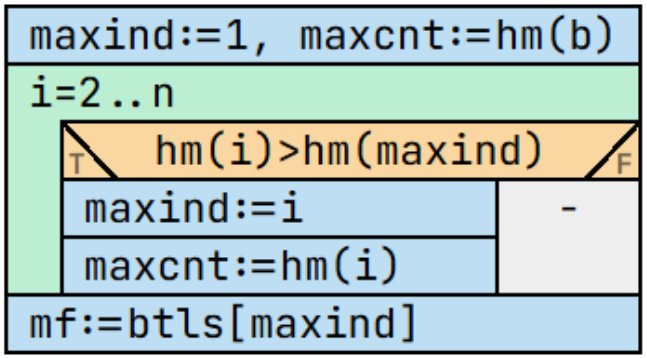
```
In: n∈N, btls∈S[1..n]
Aux: maxind∈N, maxcnt∈N
Out: mf∈S
Fn: hm:N->N, hm(i)=COUNT(j=i..n,btls[i]=btls[j])
Pre: -
Post: (maxind,maxcnt)=MAX(i=1..n,hm(i)) and
    mf=btls[maxind]
```

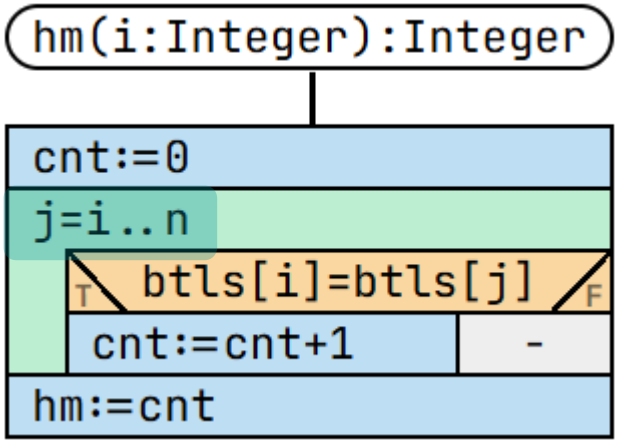
Example
most frequent value

Variation: only count the number of elements with the same value after a given element.

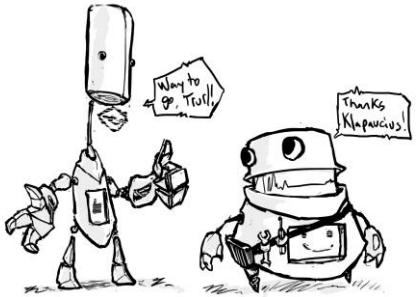
Specification:

```
In: n∈N, btls∈S[1..n]
Aux: maxind∈N, maxcnt∈N
Out: mf∈S
Fn: hm:N->N,
    hm(i)=COUNT(j=i..n,btls[i]=btls[j])
Pre: -
Post: (maxind,maxcnt)=MAX(i=1..n,hm(i)) and
    mf=btls[maxind]
```





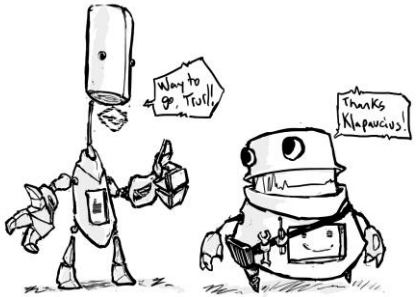
Example day of winning



Trurl and Klapaucius the two general engineers bet on their favourite dragon of probability in the dragon race every week. They record how much they win or lose. They are wondering if there has ever been a day when they won by always losing on the previous k days?

1	2	3	4	5	6	7	n=8
2	-3	4	-2	-1	5	-2	23

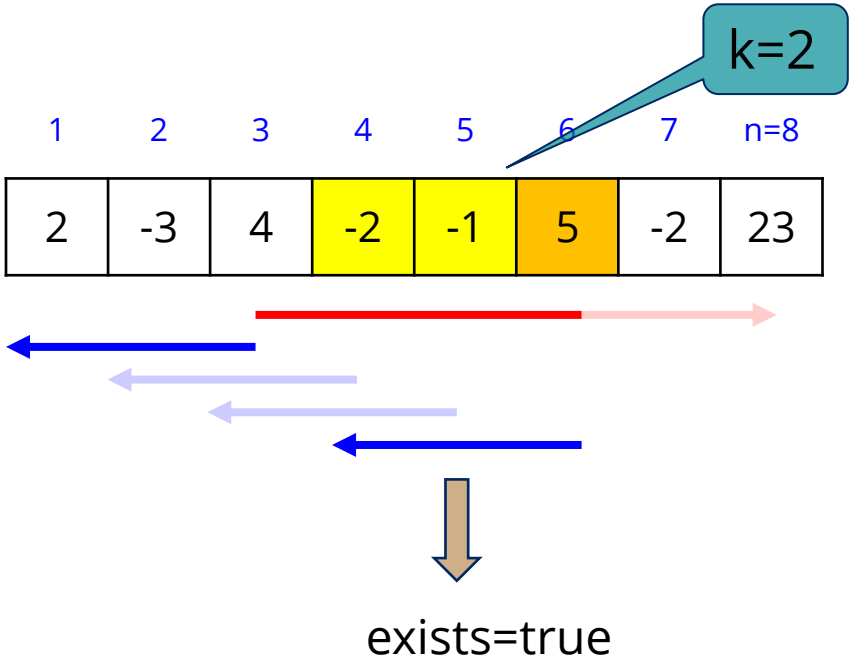
Example day of winning



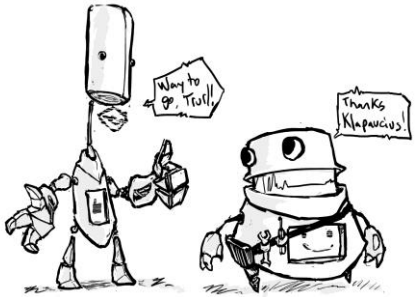
Is there any positive number in an integer sequence before which k numbers are all negative?

Steps:

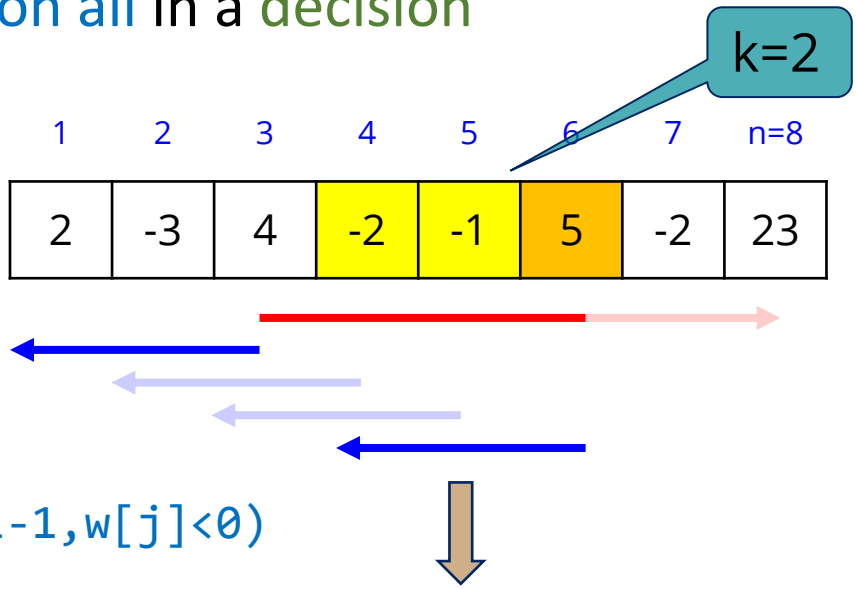
1. Decide whether exits an index (decision)
2. before which, if k numbers are considered, all are negative.
3. Decision all in a decision



Example day of winning



Is there any positive number in an integer sequence before which k numbers are all negative? Decision all in a decision



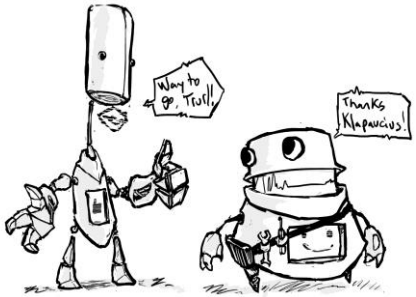
Specification:

```
In: n∈N, w∈Z[1..n], k∈N
Out: exists∈L
Fn: alllose:N->L,
    alllose(i)=FORALL(j=i-k..i-1,w[j]<0)
Pre: k>0
Post: exists=EXISTS(i=k+1..n,w[i]>0 and
                                alllose(i))
```

exists=true

Example day of winning

1	2	3	4	5	6	7	n=8
2	-3	4	-2	-1	5	-2	23



Is there any positive number in an integer sequence before which k numbers are all negative? Decision all in a decision

Decision:

```
In: b∈Z, e∈Z
Out: exists∈L
Pre: e<=u
Post: exists=EXISTS(i=b..e,A(i))
```

Decision all:

```
In: b∈Z, u∈Z
Out: all∈L
Pre: -
Post: all=FORALL(i=b..e, A(i))
```

Specification:

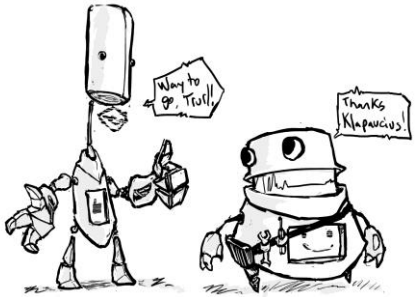
```
In: n∈N, w∈Z[1..n], k∈N
Out: exists∈L
Fn: alllose:N->L,
    alllose(i)=FORALL(j=i-k..i-1,w[j]<0)
Pre: k>0
Post: exists=EXISTS(i=k+1..n,w[i]>0 and alllose(i))
```

decision		
b..e	~	k+1..n
A(i)	~	w[i]>0 and alllose(i)

decision all		
i	~	j
b..e	~	i-k..i-1
A(i)	~	w[i]<0

Example day of winning

1	2	3	4	5	6	7	n=8
2	-3	4	-2	-1	5	-2	23



Decision

```
i:=b
i ≤ e and not A(i)
  i:=i+1
exists:=(i ≤ e)
```

decision

b..e	~	k+1..n
A(i)	~	w[i]>0 and alllose(i)

```
i:=k+1
i ≤ n and not (w[i]>0 and alllose(i))
  i:=i+1
exists:=(i ≤ n)
```

Decision all

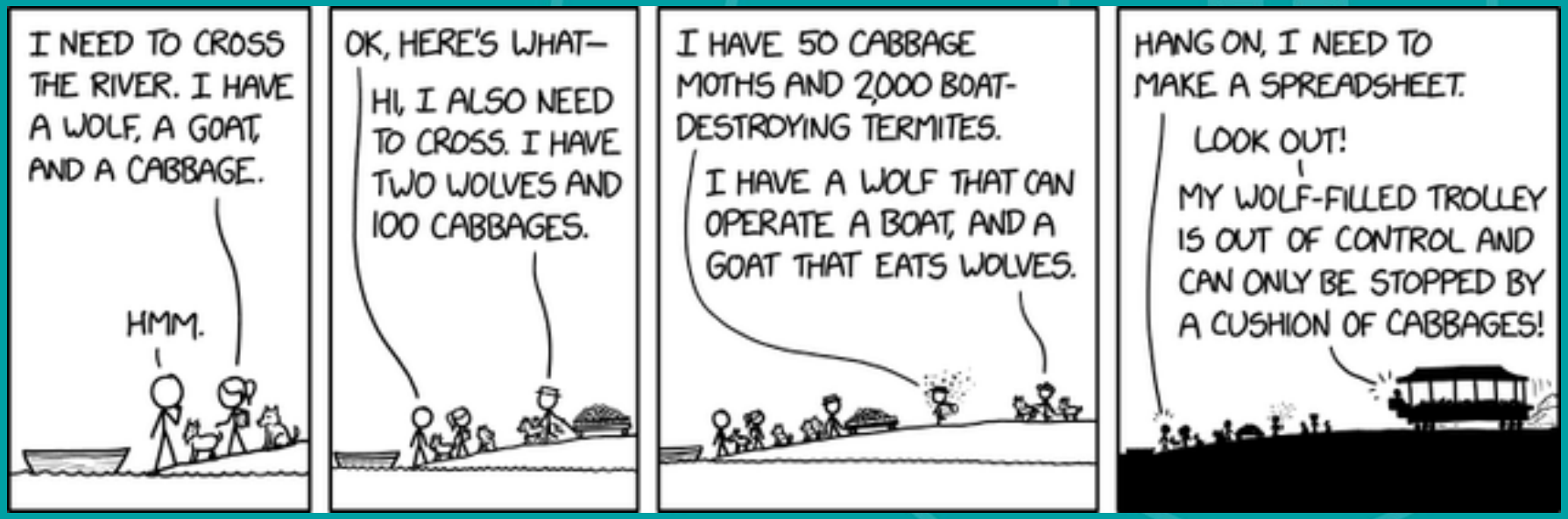
```
i:=b
i ≤ e and A(i)
  i:=i+1
all:=(i>e)
```

decision all

i	~	j
b..e	~	i-k..i-1
A(i)	~	w[i]<0

```
alllose(i:Integer):Boolean
j:=i-k
j ≤ i-1 and w[i]<0
  j:=j+1
all:=(j>i-1)
alllose:=all
```

ELTE
FACULTY OF
INFORMATICS



Thank you for your attention!